GriidAi

Export Conversion Engine

Backend Implementation Plan

Technical Specification — v1.0

Document owner **GriidAi Engineering**	Status **Ready for Implementation**
Date **March 2026**	Estimated effort **8–12 working days**

1. Overview & Objectives

This document provides a complete backend implementation plan for the GriidAi Export Conversion Engine. It covers every component from environment setup through production deployment, with actionable tasks, code patterns, and acceptance criteria for each phase.

GOAL Transition from internal-only GeoParquet/COG storage to a user-selectable multi-format export system, backed by an asynchronous conversion pipeline.

1.1 Scope

Dimension	Details
In scope	Conversion logic (Vector + Raster), /export endpoint, async task queue, temp file storage, status polling endpoint, cleanup job
Out of scope	Frontend modal (separate task), authentication changes, billing metering, permanent file storage migration
Primary stack	FastAPI · Celery · Redis · GDAL · Rasterio · GeoPandas · Fiona
Deployment target	Docker Compose (dev) → Kubernetes (prod)
Estimated effort	8–12 engineering days depending on data volume edge cases

1.2 Supported Formats

Format	Source → Target / Notes
GeoJSON (.geojson)	GeoParquet → GeoJSON via GeoPandas · web-friendly, human-readable
ESRI Shapefile (.shp)	GeoParquet → Shapefile via Fiona · column names auto-truncated to 10 chars
KML (.kml)	GeoParquet → KML via Fiona · CRS must be re-projected to EPSG:4326
Standard GeoTIFF (.tif)	COG → GeoTIFF via Rasterio · strips COG tiling, preserves CRS & metadata
PNG + world file (.png/.pgw)	COG → PNG via Rasterio · world file generated from affine transform
JPEG (.jpg)	COG → JPEG via Rasterio · alpha channel stripped, 3-band only

2. System Architecture

The export pipeline follows a request-queue-worker pattern. The API layer accepts export requests and returns immediately with a task ID. Celery workers process conversions asynchronously. Converted files are written to temporary storage and a signed URL is returned to the client upon completion.

2.1 Request Flow

Step	Component / Action
1	Client sends POST /export with { file_id, target_format }
2	/export endpoint validates input, retrieves file metadata, pushes Celery task
3	Endpoint returns 202 Accepted with { task_id, status: 'queued' }
4	Celery worker picks up task — routes to VectorWorker or RasterWorker
5	Worker reads source file (GeoParquet or COG) from internal storage
6	Worker runs conversion, writes output to /tmp/exports/
7	Output file is uploaded to object storage (S3/GCS) with a 24h signed URL
8	Task state updated to SUCCESS with { download_url }
9	Client polls GET /export/status/{task_id} until SUCCESS or FAILURE

2.2 Technology Choices

Technology	Role & Justification
FastAPI	Async-native Python API framework; clean dependency injection; OpenAPI docs auto-generated
Celery 5.x	Distributed task queue with retry logic, ETA support, and real-time state tracking
Redis 7.x	Celery broker + result backend; low latency; supports pub/sub for live status updates
GeoPandas + Fiona	High-level vector I/O; handles GeoParquet natively via PyArrow; all OGR formats
Rasterio	GDAL-backed raster I/O; windowed reading for large COG files; zero memory spikes
GDAL 3.7+	Underlying driver for all GIS format support; must be present in all worker containers
S3 / GCS + presigned URLs	Temp file hosting with automatic expiry; avoids serving files directly from workers

3. Environment & Dependency Setup

CRITICAL GDAL must be installed at the OS level in every container that runs a Celery worker. Installing only via pip is insufficient — native binaries are required.

3.1 Dockerfile (Worker Image)

Use a GDAL-preinstalled base image to avoid build complexity:

```
FROM ghcr.io/osgeo/gdal:ubuntu-full-3.7.0

RUN apt-get update && apt-get install -y \
    python3-pip python3-dev libgeos-dev libproj-dev \
    libspatialindex-dev && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . /app
WORKDIR /app
CMD ["celery", "-A", "app.celery_app", "worker", "--loglevel=info"]
```

3.2 Python Dependencies (requirements.txt)

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
celery[redis]==5.4.0
redis==5.0.4
geopandas==0.14.4
fiona==1.9.6
rasterio==1.3.10
pyarrow==16.0.0
boto3==1.34.0          # S3 — swap for google-cloud-storage if using GCS
pydantic==2.7.0
python-multipart==0.0.9
```

3.3 Docker Compose (Development)

```
services:
  api:
    build: .
    command: uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
    ports: ['8000:8000']
    env_file: .env

  worker:
    build: .
    command: celery -A app.celery_app worker --loglevel=info -Q exports
    env_file: .env
    depends_on: [redis]

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
```

3.4 Environment Variables (.env)

Variable	Purpose / Example value
REDIS_URL	redis://redis:6379/0 — Celery broker
CELERY_RESULT_BACKEND	redis://redis:6379/1 — Task state store
INTERNAL_STORAGE_PATH	/data/storage — Root path for GeoParquet/COG files
EXPORT_TMP_DIR	/tmp/exports — Local temp output before upload
S3_BUCKET	griidai-exports — Target bucket for converted files
S3_SIGNED_URL_EXPIRY	86400 — Signed URL TTL in seconds (24h)
MAX_EXPORT_SIZE_MB	512 — Reject exports larger than this before queuing

4. API Layer — FastAPI Endpoints

4.1 POST /export (initiate export)

Validates the request, checks file existence and size, then pushes the Celery task and returns 202 immediately.

Request schema

Field	Type / Validation
file_id	str — UUID, required. Must exist in internal storage.
target_format	Enum: geojson | shapefile | kml | geotiff | png | jpeg

Response schema (202 Accepted)

Field	Type / Notes
task_id	str — Celery task UUID. Used for polling.
status	str — Always "queued" at this point.
estimated_seconds	int — Rough estimate based on file size class.

Validation rules

- File must exist in internal storage path. Return 404 if not found.
- File type (vector vs. raster) must be compatible with target_format. Return 400 with a clear message if mismatched — e.g. cannot export GeoParquet as GeoTIFF.
- File size must be below MAX_EXPORT_SIZE_MB. Return 413 if exceeded.
- Rate-limit: max 5 concurrent export tasks per user (check active Celery tasks).

FastAPI handler pattern

```
@router.post('/export', status_code=202)
async def initiate_export(req: ExportRequest, db: Session = Depends(get_db)):
    file_meta = get_file_metadata(req.file_id, db)
    if not file_meta:
        raise HTTPException(404, 'File not found')
    validate_format_compatibility(file_meta.file_type, req.target_format)
    validate_file_size(file_meta.size_mb)

    task = export_task.apply_async(
        args=[req.file_id, req.target_format],
        queue='exports'
    )
    return {'task_id': task.id, 'status': 'queued',
            'estimated_seconds': estimate_duration(file_meta.size_mb)}
```

4.2 GET /export/status/{task_id}

Polls the Celery result backend for task state. The frontend calls this endpoint every 3 seconds until it receives SUCCESS or FAILURE.

Response schema by state

State	Response fields
QUEUED	{ status: "queued", progress: 0 }
PROGRESS	{ status: "progress", progress: 0–100, message: "Converting..." }
SUCCESS	{ status: "success", progress: 100, download_url: "https://...", expires_at: "..." }
FAILURE	{ status: "failure", error: "Readable error message" }

```
@router.get('/export/status/{task_id}')
async def export_status(task_id: str):
    result = AsyncResult(task_id, app=celery_app)
    state = result.state
    meta  = result.info or {}

    if state == 'PENDING':
        return {'status': 'queued', 'progress': 0}
    elif state == 'PROGRESS':
        return {'status': 'progress', **meta}
    elif state == 'SUCCESS':
        return {'status': 'success', **result.get()}
    else:
        return {'status': 'failure', 'error': str(meta.get('exc_message',
'Unknown error'))}
```

5. Celery & Redis Configuration

5.1 Celery Application

```
from celery import Celery
import os

celery_app = Celery(
    'griidai_exports',
    broker=os.environ['REDIS_URL'],
    backend=os.environ['CELERY_RESULT_BACKEND'],
    include=['app.tasks.export_tasks']
)
```

```
celery_app.conf.update(
    task_serializer='json',
    result_serializer='json',
    accept_content=['json'],
    task_acks_late=True,            # re-queue on worker crash
    task_reject_on_worker_lost=True,
    worker_prefetch_multiplier=1,   # one task per worker at a time
    result_expires=86400,           # 24h result TTL in Redis
    task_routes={'app.tasks.*': {'queue': 'exports'}},
)
```

5.2 Task Retry Policy

Setting	Value / Rationale
max_retries	3 — transient I/O failures (S3 timeout, Redis blip)
retry_backoff	True — exponential backoff: 2s, 4s, 8s between retries
retry_jitter	True — prevents thundering herd on mass failures
soft_time_limit	600s — graceful cancellation after 10 minutes
time_limit	660s — hard kill after 11 minutes if soft limit ignored

```
@celery_app.task(
    bind=True,
    max_retries=3,
    soft_time_limit=600,
    time_limit=660,
    autoretry_for=(IOError, OSError),
    retry_backoff=True,
    retry_jitter=True,
)
def export_task(self, file_id: str, target_format: str):
    ...
```

6. Vector Conversion Worker

6.1 Routing Logic

The export_task function inspects the file's metadata to decide which conversion path to use:

```
def export_task(self, file_id, target_format):
    meta = get_file_metadata(file_id)
    if meta.file_type == 'vector':
        return run_vector_conversion(self, file_id, target_format)
    elif meta.file_type == 'raster':
        return run_raster_conversion(self, file_id, target_format)
```

```
    else:
        raise ValueError(f'Unknown file type: {meta.file_type}')
```

6.2 GeoParquet → GeoJSON

```
import geopandas as gpd

def convert_to_geojson(input_path, output_path, task):
    task.update_state(state='PROGRESS', meta={'progress': 10, 'message':
'Reading GeoParquet...'})
    gdf = gpd.read_parquet(input_path)

    task.update_state(state='PROGRESS', meta={'progress': 50, 'message':
'Writing GeoJSON...'})
    gdf.to_file(output_path + '.geojson', driver='GeoJSON')
    return output_path + '.geojson'
```

6.3 GeoParquet → ESRI Shapefile

```
def convert_to_shapefile(input_path, output_path, task):
    task.update_state(state='PROGRESS', meta={'progress': 10, 'message':
'Reading GeoParquet...'})
    gdf = gpd.read_parquet(input_path)

    # Shapefile column names: max 10 characters
    gdf.columns = [c[:10] for c in gdf.columns]

    # Truncate strings > 254 chars (DBF limitation)
    for col in gdf.select_dtypes(include='object').columns:
        gdf[col] = gdf[col].str[:254]

    task.update_state(state='PROGRESS', meta={'progress': 50, 'message':
'Writing Shapefile...'})
    gdf.to_file(output_path + '.shp', driver='ESRI Shapefile')
    return output_path + '.shp'
```

6.4 GeoParquet → KML

NOTE KML requires CRS = WGS84 (EPSG:4326). Always re-project before writing, even if the source claims to already be EPSG:4326.

```
def convert_to_kml(input_path, output_path, task):
    task.update_state(state='PROGRESS', meta={'progress': 10, 'message':
'Reading GeoParquet...'})
    gdf = gpd.read_parquet(input_path)
```

```
    task.update_state(state='PROGRESS', meta={'progress': 35, 'message':
'Reprojecting to WGS84...'})
    gdf = gdf.to_crs(epsg=4326)

    task.update_state(state='PROGRESS', meta={'progress': 60, 'message':
'Writing KML...'})
    gdf.to_file(output_path + '.kml', driver='KML')
    return output_path + '.kml'
```

6.5 Large File Handling (Chunked Reading)

For GeoParquet files exceeding 200 MB, use PyArrow's native chunked reader to avoid loading the full dataset into memory:

```
import pyarrow.parquet as pq
import pyarrow as pa

def chunked_read_geoparquet(input_path, chunk_rows=50_000):
    pf = pq.ParquetFile(input_path)
    for batch in pf.iter_batches(batch_size=chunk_rows):
        gdf = gpd.GeoDataFrame.from_arrow(batch)
        yield gdf
```

7. Raster Conversion Worker

7.1 COG → Standard GeoTIFF

A Cloud Optimized GeoTIFF has internal tiling and overviews that are not needed in the user-facing export. The conversion strips these and writes a flat GeoTIFF that opens in any GIS tool:

```
import rasterio
from rasterio.enums import Compression

def convert_to_geotiff(input_path, output_path, task):
    with rasterio.open(input_path) as src:
        profile = src.profile.copy()
        profile.update(
            driver='GTiff',
            tiled=False,
            compress='lzw',   # lossless, widely supported
            interleave='band'
        )
        task.update_state(state='PROGRESS', meta={'progress': 20, 'message':
'Reading COG...'})
        # Windowed reading — avoids loading entire file into RAM
        with rasterio.open(output_path + '.tif', 'w', **profile) as dst:
            for ji, window in src.block_windows(1):
                dst.write(src.read(window=window), window=window)
    return output_path + '.tif'
```

7.2 COG → PNG + World File

```
def convert_to_png(input_path, output_path, task):
    with rasterio.open(input_path) as src:
        profile = src.profile.copy()
        profile.update(driver='PNG', compress='deflate')

        task.update_state(state='PROGRESS', meta={'progress': 30, 'message':
'Writing PNG...'})
        with rasterio.open(output_path + '.png', 'w', **profile) as dst:
            dst.write(src.read())
            dst.write_transform = src.transform

        # World file (.pgw) — georeferencing for PNG
        t = src.transform
        with open(output_path + '.pgw', 'w') as wf:
            wf.write(f'{t.a}\n{t.d}\n{t.b}\n{t.e}\n{t.c}\n{t.f}\n')
    return [output_path + '.png', output_path + '.pgw']
```

7.3 COG → JPEG

NOTE

JPEG does not support alpha channel or more than 3 bands. Strip band 4 (alpha) before writing. If fewer than 3 bands exist, raise a clear error rather than producing a corrupt file.

```
def convert_to_jpeg(input_path, output_path, task):
    with rasterio.open(input_path) as src:
        band_count = min(src.count, 3)   # take RGB only
        if src.count == 0:
            raise ValueError('Source has no bands to export as JPEG')

        profile = src.profile.copy()
        profile.update(driver='JPEG', count=band_count, compress='jpeg')
        profile.pop('nodata', None)   # JPEG has no nodata concept

        task.update_state(state='PROGRESS', meta={'progress': 30, 'message':
'Writing JPEG...'})
        with rasterio.open(output_path + '.jpg', 'w', **profile) as dst:
            dst.write(src.read(list(range(1, band_count + 1))))
    return output_path + '.jpg'
```

8. Temporary Storage & Signed URLs

8.1 Upload Flow

Step	Action
1	Worker writes converted file to EXPORT_TMP_DIR on local disk
2	Worker calls upload_to_storage(local_path, file_id, format) — uploads to S3/GCS
3	A presigned URL with 24h TTL is generated and returned to the task result
4	Local temp file is deleted immediately after upload succeeds

8.2 S3 Upload Helper

```
import boto3, os, uuid
from datetime import datetime, timedelta

s3 = boto3.client('s3')
BUCKET = os.environ['S3_BUCKET']
TTL    = int(os.environ.get('S3_SIGNED_URL_EXPIRY', 86400))

def upload_to_storage(local_path: str, file_id: str, fmt: str) -> dict:
    key = f'exports/{file_id}/{uuid.uuid4().hex}.{fmt}'
    s3.upload_file(local_path, BUCKET, key)
    os.remove(local_path)   # clean up immediately

    url = s3.generate_presigned_url(
        'get_object',
        Params={'Bucket': BUCKET, 'Key': key},
        ExpiresIn=TTL
    )
    return {
        'download_url': url,
        'expires_at': (datetime.utcnow() + timedelta(seconds=TTL)).isoformat()
    }
```

8.3 Automatic Cleanup Job

Use Celery Beat to run a nightly cleanup that removes orphaned temp files and expired S3 objects:

```
celery_app.conf.beat_schedule = {
    'cleanup-expired-exports': {
        'task': 'app.tasks.cleanup.purge_expired_exports',
        'schedule': crontab(hour=3, minute=0),  # 3am daily
    },
```

```
    }
```

9. Error Handling & Observability

9.1 Error Taxonomy

Error type	HTTP code / Celery state / Action
File not found	404 — rejected at API layer before queuing
Format mismatch (vector→raster)	400 — rejected at API layer before queuing
File too large	413 — rejected at API layer before queuing
Conversion failure (corrupt source)	Task → FAILURE · error message returned to client
Worker timeout (>10 min)	Task → FAILURE · SoftTimeLimitExceeded caught and re-raised as readable error
S3 upload failure	Task → RETRY (up to 3x) then FAILURE
GDAL not available	Worker startup fails · Docker healthcheck catches this before traffic is routed

9.2 Structured Logging

Every conversion step emits a structured JSON log line. Use these fields consistently:

Field	Value / Notes
task_id	Celery task UUID — use for end-to-end tracing
file_id	Source file identifier
target_format	e.g. geojson, geotiff
step	read | reproject | write | upload | cleanup
duration_ms	Wall time for this step — flag if > 30s
file_size_mb	Source file size — correlate with duration for performance tracking
error	Full exception string on failure — include stack trace

RECOMMENDATION Add Sentry or OpenTelemetry tracing from Day 1. Conversion failures in rasterio/GDAL are notoriously cryptic — distributed traces save hours of debugging.

10. Implementation Phases

Phase 1 — Environment & Scaffolding — Days 1–2

- Set up GDAL base Docker image for worker container
- Install and verify: geopandas, rasterio, fiona, pyarrow, celery, redis
- Write a smoke test that reads a sample GeoParquet and COG from disk
- Configure Celery app with Redis broker/backend
- Create Docker Compose with api, worker, redis services
- Verify worker starts and accepts tasks from the queue
- Set up .env file and document all required variables

Phase 2 — Vector Conversion Logic — Days 3–4

- Implement convert_to_geojson with progress callbacks
- Implement convert_to_shapefile with column name truncation and string truncation
- Implement convert_to_kml with automatic CRS reprojection to EPSG:4326
- Add chunked reader for large GeoParquet files (>200 MB threshold)
- Write unit tests for each converter — use 3 test datasets: small (<1 MB), medium (50 MB), large (300 MB)
- Test edge cases: empty GeoDataFrame, mixed geometry types, non-UTF-8 column names
- Verify Shapefile output opens correctly in QGIS and ArcGIS

Phase 3 — Raster Conversion Logic — Days 5–6

- Implement convert_to_geotiff with windowed reading and LZW compression
- Implement convert_to_png with world file (.pgw) generation
- Implement convert_to_jpeg with 3-band enforcement and nodata stripping
- Add pre-flight check: reject JPEG conversion if source has <1 band
- Write unit tests for each converter — validate with rasterio.open() on output
- Test large COG (2 GB): verify windowed reading does not spike memory above 512 MB
- Verify GeoTIFF output opens with correct CRS in QGIS

Phase 4 — API Endpoints & Async Wiring — Days 7–8

- Implement POST /export — validation, routing, task dispatch, 202 response

- Implement GET /export/status/{task_id} — state polling with typed response
- Add format-compatibility validation (vector formats reject COG, raster formats reject GeoParquet)
- Add file size guard — 413 error if file exceeds MAX_EXPORT_SIZE_MB
- Wire export_task Celery function to call correct vector/raster converter
- Wire upload_to_storage after conversion — include download_url in SUCCESS result
- Integration test: POST /export → poll /export/status → download the file

Phase 5 **Storage, Cleanup & Hardening** **Days 9–10**

- Implement S3 upload helper with presigned URL generation
- Ensure local temp file is deleted after successful upload
- Implement Celery Beat nightly cleanup task for orphaned S3 objects
- Add structured logging to all conversion steps (task_id, file_id, step, duration_ms)
- Add Sentry error reporting (or equivalent)
- Configure worker concurrency: 2 workers per machine (GDAL is CPU-bound)
- Load test: 20 concurrent export requests — verify queue drains correctly, no timeouts
- Write Kubernetes deployment manifest for worker pods with GDAL base image

11. Acceptance Criteria

11.1 Per-format checks

Format	Acceptance test
GeoJSON	Output opens in geojson.io — geometries and attributes visible. CRS preserved.
Shapefile	Output opens in QGIS. All column names ≤ 10 chars. No truncated geometry.
KML	Output opens in Google Earth. Geometries correct. CRS is WGS84.
GeoTIFF	rasterio.open() returns same CRS and band count as source. File opens in QGIS.
PNG + world file	PNG renders. .pgw file exists. Loading PNG + .pgw in QGIS places it correctly.
JPEG	File is exactly 3 bands. No alpha channel. No nodata artifacts. Opens in standard image viewers.

11.2 System-level checks

- POST /export returns 202 within 200 ms regardless of file size
- GET /export/status returns within 50 ms (Redis read)
- A 50 MB GeoParquet → GeoJSON conversion completes within 60 seconds
- A 500 MB COG → GeoTIFF conversion completes within 5 minutes via windowed reading
- Worker memory does not exceed 512 MB during large file conversion
- Signed URL is valid and downloadable immediately after SUCCESS state is returned
- FAILURE state returns a human-readable error message, not a Python traceback
- Celery worker recovers and re-queues task if the process is killed mid-conversion

12. Risks & Mitigations

Risk	Likelihood / Impact / Mitigation
GDAL version mismatch between dev and prod	High / High — Pin GDAL version in base image tag. Never use 'latest'.
Very large COG files (>2 GB) causing OOM	Medium / High — Windowed reading + worker memory limit (512 MB). Reject if source exceeds MAX_EXPORT_SIZE_MB.
KML reprojection failing on unusual CRS	Medium / Medium — Catch proj exceptions explicitly. Return 400 with CRS info in error message.
S3 signed URL expiry too short for slow downloads	Low / Medium — Default to 24h TTL. Expose as configurable env var so ops can adjust without redeploy.
Celery task queue building up (worker starved)	Low / High — Monitor queue depth in Grafana. Auto-scale worker pods when depth > 20.
Shapefile column name collision after truncation	Medium / Medium — After truncating to 10 chars, deduplicate by appending a numeric suffix (col1, col2).

End of document — GriidAi Engineering